

Unit-3 Overview of Java Script

3.1 Overview of Client & Server-Side Scripting

On a dynamic website there are client -side and server -side scripts. Client-side and server-side are sometimes referred to as front-end and back-end. The client-side of a website refers to the web browser and the server-side is where the data and source code is stored.

For example, developers can use JavaScript, a client-side programming language, to create forms that collect user input. Server-side development can process this input. For example, developers can use PHP to connect a database to a website and send user-inputted data to the database.

3.2 Structure of Java Script

JavaScript Introduction

- JavaScript is the world's most popular programming language.
- JavaScript is the programming language of the Web.
- JavaScript is easy to learn.
- You don't have to get or download JavaScript.
- JavaScript is already running in your browser on your computer, on your tablet, and on your smart-phone.
- JavaScript is free to use for everyone.
- JavaScript is one of the 3 languages all web developers must learn:
 1. HTML to define the content of web pages
 2. CSS to specify the layout of web pages
 3. JavaScript to program the behavior of web pages

Where we insert JavaScript?

You can place any number of scripts in an HTML document.

Scripts can be placed in the <body>, or in the <head> section of an HTML page, or in both.

The <script> Tag in <body>

In HTML, JavaScript code is inserted between <script> and </script> tags.

Example:

```
<html>
<body>
<h2>JavaScript in Body</h2>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
</body>
</html>
```

JavaScript in <head>

Example:

```
<html>
<head>
<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>
<p id="demo"></p>
</body>
</html>
```

Unit-3 Overview of Java Script

```
</script>
</head>
<body>
<h2>Demo JavaScript in Head</h2>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
</html>
```

External JavaScript

External scripts are practical when the same code is used in many different web pages.

JavaScript files have the file extension .js

To use an external script, put the name of the script file in the src (source) attribute of a <script> tag:

You can place an external script reference in <head> or <body> as you like.

The script will behave as if it was located exactly where the <script> tag is located.

External scripts cannot contain <script>tags.

myScript.js

```
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
```

External_js.html

```
<html>
<head>
</head>
<body>
<h2>Demo External JavaScript</h2>
<p id="demo">A Paragraph.</p>
<button type="button" onclick="myFunction()">Try it</button>
<p>This example links to "myScript.js".</p>
<p>(myFunction is stored in "myScript.js")</p>
<script src="myScript.js"></script>
</body>
</html>
```

External JavaScript Advantages

Placing scripts in external files has some advantages:

- It separates HTML and code
- It makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files can speed up page loads

To add several script files to one page - use several script tags:

Example:

```
<script src="myscript1.js"></script>
<script src="myscript2.js"></script>
```

External References

An external script can be referenced in 3 different ways:

- With a full URL (a full web address)

Unit-3 Overview of Java Script

- With a file path (like /js/)
- Without any path

Example:

```
<script src="https://www.w3schools.com/js/myScript.js"></script>
<script src="/js/myScript.js"></script>
<script src="myScript.js"></script>
```

JavaScript Syntax

// How to create variables:

```
var x;
let y;
```

// How to use variables:

```
x = 5;
y = 6;
let z = x + y;
```

JavaScript Comments

Single Line Comments

Single line comments start with //.

Any text between // and the end of the line will be ignored by JavaScript (will not be executed).

Multi-line Comments

Multi-line comments start with /* and end with */.

Any text between /* and */ will be ignored by JavaScript.

```
/*
The code below will change
the heading with id = "myH"
and the paragraph with id = "myP"
in my web page:
*/
```

```
<script>
//document.getElementById("myH").innerHTML = "My First Page";
document.getElementById("myP").innerHTML = "My first paragraph.";
</script>
<p>The line starting with // is not executed.</p>
```

3.3 Data types and Variables

JavaScript Variables

Variables are containers for storing data (storing data values).

Following are the ways to declare JavaScript variables:

1. var
2. let
3. const
4. Using nothing

In this example, x, y, and z, are variables, declared with the var keyword:

If you want your code to run in older version, you must use var.

Example:

```
<p id="demo"></p>
```

Unit-3 Overview of Java Script

```
<script>
var x = 5;
var y = 6;
var z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>
```

If you think the value of the variable can change, use let.

In this example, x, y, and z, are variables, declared with the let keyword:

Example:

```
<p id="demo"></p>
<script>
let x = 5;
let y = 6;
let z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>
```

If you don't want to change value always declare variables with const.

Example:

```
<p id="demo"></p>
<script>
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo").innerHTML =
"The total is: " + total;
</script>
```

In this example, x, y, and z, are undeclared variables:

Example:

```
<p id="demo"></p>
<script>
x = 5;
y = 6;
z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>
```

JavaScript Identifiers

- All JavaScript **variables** must be **identified** with **unique names**.
- These unique names are called **identifiers**.

Unit-3 Overview of Java Script

- Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter
- Names can also begin with \$ and _ .
- Names are case sensitive (y and Y are different variables)
- Reserved words (like JavaScript keywords) cannot be used as names

Data types:

- In programming, data types are an important concept.
- To be able to operate on variables, it is important to know something about the type.
- Without data types, a computer cannot safely solve this:
- JavaScript variables can hold different data types: numbers, strings and objects.

```
let length = 16;           // Number
let lastName = "Johnson"; // String
let x = {firstName:"John", lastName:"Doe"}; // Object
```

- When adding a number and a string, JavaScript will treat the number as a string.

Example:

```
<html>
<body>
<h2>JavaScript</h2>
<p>When adding a number and a string, JavaScript will treat the number as a string.</p>
<p id="demo"></p>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>
<p id="demo4"></p>
<p id="demo5"></p>
<script>
let x = 16 + "Volvo";
document.getElementById("demo").innerHTML = x;
</script>
<script>
let y = "16" + "volvo";
document.getElementById("demo1").innerHTML = y;
</script>
<script>
let z = "Volvo" + 16;
document.getElementById("demo2").innerHTML = z;
</script>
<script>
let w = 16 + 4 + "Volvo";
document.getElementById("demo3").innerHTML = w;
```

Unit-3 Overview of Java Script

//In this example, JavaScript treats 16 and 4 as numbers, until it reaches "Volvo".

```
</script>
```

```
<script>
```

```
let a = "Volvo" + 16 + 4;
```

```
document.getElementById("demo4").innerHTML = a;
```

//In this example, since the first operand is a string, all operands are treated as strings.

```
</script>
```

```
<script>
```

```
let b = 16 + 16;
```

```
document.getElementById("demo5").innerHTML = b;
```

```
</script>
```

```
</body>
```

```
</html>
```

- JavaScript has dynamic types. This means that the same variable can be used to hold different data types:

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
let x;      // Now x is undefined
```

```
x = 5;      // Now x is a Number
```

```
x = "John"; // Now x is a String
```

```
document.getElementById("demo").innerHTML = x;
```

```
</script>
```

JavaScript Strings

- A string (or a text string) is a series of characters like "Swaraaj Desai".
- Strings are written with quotes. You can use single or double quotes.
- You can use quotes inside a string, as long as they don't match the quotes surrounding the string.

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
let carName1 = "BMW";
```

```
let carName2 = 'Audi';
```

```
document.getElementById("demo").innerHTML = carName1 + "<br>" + carName2;
```

```
</script>
```

Example:

```
let answer1 = "It's alright";      // Single quote inside double quotes
```

```
let answer2 = "He is called 'Johnny'"; // Single quotes inside double quotes
```

```
let answer3 = 'He is called "Johnny"'; // Double quotes inside single quotes
```

JavaScript Numbers

- JavaScript has only one type of numbers.
- Numbers can be written with, or without decimals:
- Extra large or extra small numbers can be written with scientific (exponential) notation

Example:

Unit-3 Overview of Java Script

```
<p id="demo"></p>
<script>
let x1 = 34.00;
let x2 = 34;
let x3 = 3.14;
let y = 123e5;    // 123000000
let z = 123e-5;   // 0.00123
document.getElementById("demo").innerHTML =x1 + "<br>" + x2 + "<br>" + x3 + "<br>" + y
+ "<br>" + z;
</script>
```

JavaScript Booleans

Booleans can only have two values: true or false.

Example:

```
<p id="demo"></p>
<script>
let x = 5;
let y = 5;
let z = 6;
document.getElementById("demo").innerHTML =(x == y) + "<br>" + (x == z);
</script>
```

JavaScript Arrays

JavaScript arrays are written with square brackets.

Array items are separated by commas.

Array indexes are zero-based, which means the first item is [0], second is [1], and so on.

The following code declares (creates) an array called cars, containing three items (car names):

Example:

```
<p id="demo"></p>
<script>
const cars = ["Saab","Volvo","BMW"];
document.getElementById("demo").innerHTML = cars[0];
</script>
```

JavaScript Objects

JavaScript objects are written with curly braces {}.

Object properties are written as name:value pairs, separated by commas.

Example:

```
<p id="demo"></p>
<script>
const person = {
  firstName : "John",
  lastName  : "Doe",
  age       : 50,
  eyeColor  : "blue"
};
```

Unit-3 Overview of Java Script

```
document.getElementById("demo").innerHTML = person.firstName + " is " + person.age + "
years old.";
</script>
```

The typeof Operator

You can use the JavaScript typeof operator to find the type of a JavaScript variable. The typeof operator returns the type of a variable or an expression.

Example:

```
<p>The typeof operator returns the type of a variable or an expression.</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
typeof "" + "<br>" +
typeof "John" + "<br>" +
typeof "John Doe";
</script>
<p id="demo1"></p>
<script>
document.getElementById("demo1").innerHTML =
typeof 0 + "<br>" +
typeof 314 + "<br>" +
typeof 3.14 + "<br>" +
typeof (3) + "<br>" +
typeof (3 + 4);
</script>
```

Undefined

In JavaScript, a variable without a value has the value undefined. The type is also undefined. Any variable can be emptied, by setting the value to undefined. The type will also be undefined.

Example:

```
<h2>JavaScript</h2>
<p>The value (and the data type) of a variable with no value is <b>undefined</b>.</p>
<p id="demo"></p>
<script>
let car;
document.getElementById("demo").innerHTML =
car + "<br>" + typeof car;
</script>
<p id="demo1"></p>
<script>
let car1 = "Volvo";
car1 = undefined;
document.getElementById("demo1").innerHTML = car1 + "<br>" + typeof car1;
</script>
```

Empty Values

Unit-3 Overview of Java Script

An empty value has nothing to do with undefined.

An empty string has both a legal value and a type.

Example:

<p>An empty string has both a legal value and a type:</p>

<p id="demo"></p>

<script>

let car = "";

document.getElementById("demo").innerHTML = "The value is: " + car + "
" +

"The type is: " + typeof car;

</script>

3.4 Operators (Arithmetic, Assignment, Comparison, Logical and Conditional Operator)

❖ **Arithmetic Operators:**

Arithmetic operators are used to perform arithmetic on numbers:

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation
/	Division
%	Modulus (Division Reminder)
++	Increment
--	Decrement

Example:

<h2>JavaScript All Arithmetic</h2>

<p id="demo"></p>

<script>

let x = 5;

let y = 2;

let z = x + y;

document.getElementById("demo").innerHTML = z;

</script>

<p id="demo1"></p>

<script>

let a = 5;

let b = 2;

let c = x - y;

document.getElementById("demo1").innerHTML = c;

</script>

<p id="demo2"></p>

<script>

Unit-3 Overview of Java Script

```
let d = 5;
let e = 2;
let f = x * y;
document.getElementById("demo2").innerHTML = f;
</script>
```

```
<p id="demo3"></p>
<script>
let g = 5;
let h = 2;
let i = x / y;
document.getElementById("demo3").innerHTML = i;
</script>
```

```
<p id="demo4"></p>
<script>
let j = 5;
let k = 2;
let l = x % y;
document.getElementById("demo4").innerHTML = l;
</script>
```

```
<p id="demo5"></p>
<script>
let m = 5;
m++;
let n = m;
document.getElementById("demo5").innerHTML = n;
</script>
```

```
<p id="demo6"></p>
<script>
let o = 5;
o--;
let p = o;
document.getElementById("demo6").innerHTML = p;
</script>
```

```
<p id="demo7"></p>
<script>
let q = 5;
document.getElementById("demo7").innerHTML = q ** 2;
</script>
```

```
<p id="demo8"></p>
<script>
```

Unit-3 Overview of Java Script

```
document.getElementById("demo8").innerHTML = 100 + 50 * 3;  
</script>
```

Assignment Operators:

Assignment operators assign values to JavaScript variables.

Operator	Description	Same as
=	x=y	x=y
+=	x+=y	x=x+y
-=	x-=y	x=x-y
=	x=y	x=x*y
/=	x/=y	x=x/y
%=	x%=y	x=x%y
=	x=y	x=x**y

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
let x = 10;
```

```
document.getElementById("demo").innerHTML = x;
```

```
</script>
```

```
<p id="demo1"></p>
```

```
<script>
```

```
let y = 10;
```

```
y -= 5;
```

```
document.getElementById("demo1").innerHTML = y;
```

```
</script>
```

```
<p id="demo2"></p>
```

```
<script>
```

```
let z = 10;
```

```
z += 5;
```

```
document.getElementById("demo2").innerHTML = z;
```

```
</script>
```

```
<p id="demo3"></p>
```

```
<script>
```

```
let a = 10;
```

```
a *= 5;
```

```
document.getElementById("demo3").innerHTML = a;
```

```
</script>
```

```
<p id="demo4"></p>
```

```
<script>
```

```
let b = 10;
```

```
b /= 5;
```

```
document.getElementById("demo4").innerHTML = b;
```

Unit-3 Overview of Java Script

</script>

<p id="demo5"></p>

<script>

let c = 10;

c %= 5;

document.getElementById("demo5").innerHTML = c;

</script>

❖ Comparison Operators:

Comparison operators are used in logical statements to determine equality or difference between variables or values.

Operator	Description
==	Equal to
===	Equal value and equal type
!=	Not equal
!==	Not equal value or not equal type
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

<h2>JavaScript Comparison Operators</h2>

<p id="demo"></p>

<script>

let x = 5;

document.getElementById("demo").innerHTML = (x == "5");

</script>

<p id="demo1"></p>

<script>

let y = 5;

document.getElementById("demo1").innerHTML = (y === 5);

</script>

<p id="demo2"></p>

<script>

let z = 5;

document.getElementById("demo2").innerHTML = (z != 5);

</script>

<p id="demo3"></p>

<script>

let a = 5;

Unit-3 Overview of Java Script

```
document.getElementById("demo").innerHTML = (a !== 5);  
</script>
```

```
<p id="demo4"></p>  
<script>  
let b = 5;  
document.getElementById("demo4").innerHTML = (b > 8);  
</script>
```

```
<p id="demo5"></p>  
<script>  
let c = 5;  
document.getElementById("demo5").innerHTML = (c < 8);  
</script>
```

```
<p id="demo6"></p>  
<script>  
let d = 5;  
document.getElementById("demo6").innerHTML = (d >= 8);  
</script>
```

```
<p id="demo7"></p>  
<script>  
let e = 5;  
document.getElementById("demo7").innerHTML = (e <= 8);  
</script>
```

❖ Logical Operators

Logical operators are used to determine the logic between variables or values.

Operator	Description
&&	And
	Or
!	not

Example:

```
<h2>JavaScript Logical Operators</h2>  
<p id="demo"></p>  
<script>  
let x = 6;  
let y = 3;  
document.getElementById("demo").innerHTML =  
(x < 10 && x > 1) + "<br>" +  
(y < 10 && y < 1);  
</script>
```

Unit-3 Overview of Java Script

```
<p id="demo1"></p>
<script>
let a = 6;
let b = 3;
document.getElementById("demo1").innerHTML =
(a == 5 || b == 5) + "<br>" +
(a == 6 || b == 0) + "<br>" +
(a == 0 || b == 3) + "<br>" +
(a == 6 || b == 3);
</script>
```

```
<p id="demo2"></p>
<script>
let z = 6;
let w = 3;

document.getElementById("demo2").innerHTML =
!(z === w) + "<br>" +
!(z > w);
</script>
```

❖ Conditional Operators

JavaScript also contains a conditional operator that assigns a value to a variable based on some condition.

Syntax

variablename = (condition) ? value1:value2

Example:

```
<input id="age" value="18" />
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
function myFunction() {
  let age = document.getElementById("age").value;
  let voteable = (age < 18) ? "Too young":"Old enough";
  document.getElementById("demo").innerHTML = voteable + " to vote.";
}
</script>
```

3.5 Control Structure

1. If...Else

The if Statement

Use the if statement to specify a block of JavaScript code to be executed if a condition is true.

Unit-3 Overview of Java Script

Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Example:

```
<p id="demo">Good Evening!</p>  
<script>  
if (new Date().getHours() < 18) {  
    document.getElementById("demo").innerHTML = "Good day!";  
}  
</script>
```

2. The else Statement

Use the else statement to specify a block of code to be executed if the condition is false.

Syntax:

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

Example:

```
<p id="demo"></p>  
<script>  
const hour = new Date().getHours();  
let greeting;  
if (hour < 18) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}  
document.getElementById("demo").innerHTML = greeting;  
</script>
```

3. The else if Statement

Use the else if statement to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

Example:

```
<p id="demo"></p>  
<script>
```

Unit-3 Overview of Java Script

```
const time = new Date().getHours();
let greeting;
if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
document.getElementById("demo").innerHTML = greeting;
</script>
```

4. JavaScript Switch Statement

The switch statement is used to perform different actions based on different conditions. Use the switch statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

This is how it works:

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- If there is no match, the default code block is executed.

The break Keyword

- When JavaScript reaches a break keyword, it breaks out of the switch block.
- This will stop the execution inside the switch block.
- It is not necessary to break the last case in a switch block. The block breaks (ends) there anyway.
- If you omit the break statement, the next case will be executed even if the evaluation does not match the case.

The default Keyword

- The default keyword specifies the code to run if there is no case match.
- The default case does not have to be the last case in a switch block.

Example:

```
<p id="demo"></p>
<script>
```


Unit-3 Overview of Java Script

```
let day;
switch (new Date().getDay()) {
  case 0:
    day = "Sunday";
    break;
  case 1:
    day = "Monday";
    break;
  case 2:
    day = "Tuesday";
    break;
  case 3:
    day = "Wednesday";
    break;
  case 4:
    day = "Thursday";
    break;
  case 5:
    day = "Friday";
    break;
  case 6:
    day = "Saturday";
}
document.getElementById("demo").innerHTML = "Today is " + day;
</script>
```

5. The While Loop

The while loop loops through a block of code as long as a specified condition is true.

Syntax:

```
while (condition) {
  // code block to be executed
}
```

Example:

```
<p id="demo"></p>
<script>
let text = "";
let i = 0;
while (i < 10) {
  text += "<br>The number is " + i;
  i++;
}
document.getElementById("demo").innerHTML = text;
</script>
```

6. The Do While Loop

Unit-3 Overview of Java Script

The do while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax

```
do {  
    // code block to be executed  
}  
while (condition);
```

The example below uses a do while loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested

Example:

```
<p id="demo"></p>  
<script>  
let text = ""  
let i = 0;  
do {  
    text += "<br>The number is " + i;  
    i++;  
}  
while (i < 10);  
document.getElementById("demo").innerHTML = text;  
</script>
```

7. The For Loop

Syntax:

```
for (statement 1; statement 2; statement 3) {  
    // code block to be executed  
}
```

Statement 1 is executed (one time) before the execution of the code block.

Statement 2 defines the condition for executing the code block.

Statement 3 is executed (every time) after the code block has been executed.

Example:

```
<p id="demo"></p>  
<script>  
let text = "";  
for (let i = 0; i < 5; i++) {  
    text += "The number is " + i + "<br>";  
}  
document.getElementById("demo").innerHTML = text;  
</script>
```

8. The Break Statement

The break statement "jumps out" of a loop.

Example:

Unit-3 Overview of Java Script

```
<p id="demo"></p>
<script>
let text = "";
for (let i = 0; i < 10; i++) {
  if (i === 3) { break; }
  text += "The number is " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
```

9. The Continue Statement

The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

```
<p id="demo"></p>
<script>
let text = "";
for (let i = 0; i < 10; i++) {
  if (i === 3) { continue; }
  text += "The number is " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
```

3.6 Java Script String and Events

String:

A JavaScript string is zero or more characters written inside quotes.
We already discuss basic string earlier.

String Length

To find the length of a string, use the **built-in length property**

Example:

```
<p id="demo"></p>
<script>
let text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
document.getElementById("demo").innerHTML = text.length;
</script>
```

Escape Character

- Because strings must be written within quotes, JavaScript will misunderstand this string.
- let text = "We are the so-called "Vikings" from the north.";
- The string will be chopped to "We are the so-called ".
- The solution to avoid this problem is to use the **backslash escape character**.
- The backslash (\) escape character turns special characters into string characters:

Code	Result
\'	'
\"	"

Unit-3 Overview of Java Script

\\	\
\b	Back space
\f	Form feed
\n	New line
\r	Carriage return
\t	Horizontal tabulator
\v	Vertical tabulator

Example:

```
<p id="demo"></p>
<script>
let text = "We are the so-called \"Vikings\" from the north.";
document.getElementById("demo").innerHTML = text;
</script>
```

```
<p id="demo1"></p>
<script>
let text1 = 'It\'s alright.';
document.getElementById("demo1").innerHTML = text1;
</script>
```

```
<p id="demo2"></p>
<script>
let text2 = "The character \\ is called backslash.";
document.getElementById("demo2").innerHTML = text2;
</script>
```

String Functions:

1. **concat()**

concat() joins two or more strings. The concat() method can be used instead of the plus operator.

Example:

```
<p id="demo"></p>
<script>
let text1 = "Hello";
let text2 = "World!";
let text3 = text1.concat(" ",text2);
document.getElementById("demo").innerHTML = text3;
or
text = "Hello" + " " + "World!";
text = "Hello".concat(" ", "World!");
</script>
```

2. **split()**

A string can be converted to an array with the split() method

Example:

Unit-3 Overview of Java Script

```
<p id="demo"></p>
<script>
let text = "a,b,c,d,e,f";
const myArray = text.split(",");
document.getElementById("demo").innerHTML = myArray[4];
</script>
```

Example:

```
<p id="demo"></p>
<script>
let text = "Hello";
const myArr = text.split("");

text = "";
for (let i = 0; i < myArr.length; i++) {
  text += myArr[i] + "<br>"
}
document.getElementById("demo").innerHTML = text;
</script>
```

3. indexOf()

The indexOf() method returns the position of the first occurrence of a value in a string.

The indexOf() method returns -1 if the value is not found.

The indexOf() method is case sensitive.

Syntax

string.indexOf(searchvalue, start)

Parameters

Searchvalue Required. The string to search for.

Start Optional. The position to start from (default is 0).

Example:

```
<p id="demo"></p>
<script>
let text = "Hello world, welcome to the universe.";
let result = text.indexOf("welcome");
document.getElementById("demo").innerHTML = result;
</script>
```

```
<p id="demo1"></p>
<script>
let text1 = "Hello world, welcome to the universe.";
let result1 = text.indexOf("Welcome");
document.getElementById("demo1").innerHTML = result1;
</script>
```

```
<p id="demo2"></p>
<script>
```

Unit-3 Overview of Java Script

```
let text2 = "Hello world, welcome to the universe.";
document.getElementById("demo2").innerHTML = text2.indexOf("e");
</script>
```

```
<p id="demo3"></p>
<script>
let text3= "Hello world, welcome to the universe.";
document.getElementById("demo3").innerHTML = text3.indexOf("e", 5);
</script>
```

```
<p id="demo4"></p>
<script>
let text4 = "Hello world, welcome to the universe.";
document.getElementById("demo4").innerHTML = text4.indexOf("a", 5);
</script>
```

4. lastIndexOf()

The lastIndexOf() method returns the index (position) of the last occurrence of a specified value in a string.

The lastIndexOf() method searches the string from the end to the beginning.

The lastIndexOf() method returns the index from the beginning (position 0).

The lastIndexOf() method returns -1 if the value is not found.

The lastIndexOf() method is case sensitive.

Syntax

string.lastIndexOf(searchvalue, start)

Example:

```
<p id="demo"></p>
<script>
let text = "Hello planet earth, you are a great planet.";
let result = text.lastIndexOf("planet");
document.getElementById("demo").innerHTML = result;
</script>
```

```
<p id="demo1"></p>
<script>
let text1 = "Hello planet earth, you are a great planet.";
let result1 = text1.lastIndexOf("Planet");
document.getElementById("demo1").innerHTML = result1;
</script>
```

5. substring()

The substring() method extracts characters, between two indices (positions), from a string, and returns the substring.

The substring() method extracts characters from start to end (exclusive).

The substring() method does not change the original string.

If start is greater than end, arguments are swapped: (4, 1) = (1, 4).

Syntax

Unit-3 Overview of Java Script

string.substring(start, end)

Example:

```
<p id="demo"></p>
<script>
let text = "Hello world!";
let result = text.substring(1,7);
document.getElementById("demo").innerHTML = result;
</script>

<p id="demo1"></p>
<script>
let text1 = "Hello world!";
let result1 = text1.substring(2);
document.getElementById("demo1").innerHTML = result1;
</script>
```

6. trim()

The trim() method removes whitespace from both sides of a string.

The trim() method does not change the original string.

Syntax

string.trim()

Example:

```
<pre><p id="demo1"></p></pre>
<pre><p id="demo2"></p></pre>
<script>
let text = "  Hello World!";
let result = text.trim();
document.getElementById("demo1").innerHTML = text;
document.getElementById("demo2").innerHTML = result;
</script>
```

7. slice()

The slice() method extracts a part of a string.

The slice() method returns the extracted part in a new string.

The slice() method does not change the original string.

The start and end parameters specifies the part of the string to extract.

The first position is 0, the second is 1, ...

A negative number selects from the end of the string.

Syntax

string.slice(start, end)

Example:

```
<p id="demo"></p>
<script>
let text = "Hello world!";
let result = text.slice(0, 7);
```

Unit-3 Overview of Java Script

```
document.getElementById("demo").innerHTML = result;  
</script>
```

8. **replace()**

The `replace()` method searches a string for a value or a regular expression.

The `replace()` method returns a new string with the value(s) replaced.

The `replace()` method does not change the original string.

Syntax

```
string.replace(searchValue, newValue)
```

Example:

```
<p id="demo">Visit Microsoft!</p>
```

```
<script>
```

```
let text = document.getElementById("demo").innerHTML;
```

```
document.getElementById("demo").innerHTML = text.replace("Microsoft", "W3Schools");
```

```
</script>
```

```
<p id="demo">Mr Blue has a blue house and a blue car.</p>
```

```
<script>
```

```
let str = document.getElementById("demo").innerHTML;
```

```
let res = str.replace(/blue/g, "red");
```

```
document.getElementById("demo").innerHTML = res;
```

```
</script>
```

9. **charAt()**

The `charAt()` method returns the character at a specified index (position) in a string.

The index of the first character is 0, the second 1, ...

Syntax

```
string.charAt(index)
```

Example:

```
<p id="demo"></p>
```

```
<script>
```

```
let text = "HELLO WORLD";
```

```
let letter = text.charAt(6);
```

```
document.getElementById("demo").innerHTML = letter;
```

```
</script>
```

JavaScript Events

HTML events are "**things**" that happen to HTML elements.

When JavaScript is used in HTML pages, JavaScript can "**react**" on these events.

JavaScript lets you execute code when events are detected.

HTML allows event handler attributes, **with JavaScript code**, to be added to HTML elements.

With single quotes:

```
<element event='some JavaScript'>
```


Unit-3 Overview of Java Script

With double quotes:

```
<element event="some JavaScript">
```

Example:

```
<button onclick="document.getElementById('demo').innerHTML=Date()">The time is?</button>
```

```
<p id="demo"></p>
```

In the example above, the JavaScript code changes the content of the element with id="demo". In the next example, the code changes the content of its own element (using **this.innerHTML**):

Example:

```
<button onclick="this.innerHTML=Date()">The time is?</button>
```

Example:

```
<button onclick="displayDate()">The time is?</button>
```

```
<script>
```

```
function displayDate() {  
    document.getElementById("demo").innerHTML = Date();  
}
```

```
</script>
```

```
<p id="demo"></p>
```

3.6.3.1 Mouse Events:

1. onmousedown() and onmouseup()

Example:

```
<p id="myP" onmousedown="mouseDown()" onmouseup="mouseUp()">
```

Click the text! The mouseDown() function is triggered when the mouse button is pressed down over this paragraph, and sets the color of the text to red. The mouseUp() function is triggered when the mouse button is released, and sets the color of the text to green.

```
</p>
```

```
<script>
```

```
function mouseDown() {  
    document.getElementById("myP").style.color = "red";  
}  
function mouseUp() {  
    document.getElementById("myP").style.color = "green";  
}
```

```
</script>
```

2. onmouseenter() and onmouseleave()

Example:

```

```

```
<p>The function bigImg() is triggered when the user moves the mouse pointer onto the image.</p>
```

Unit-3 Overview of Java Script

<p>The function normalImg() is triggered when the mouse pointer is moved out of the image.</p>

<script>

```
function bigImg(x) {  
    x.style.height = "64px";  
    x.style.width = "64px";  
}
```

```
function normalImg(x) {  
    x.style.height = "32px";  
    x.style.width = "32px";  
}
```

</script>

3. onmousemove() and onmouseout()

Example:

<head>

<style>

```
div {  
    width: 200px;  
    height: 100px;  
    border: 1px solid black;  
}
```

</style>

</head>

<body>

<div onmousemove="myFunction(event)" onmouseout="clearCoor()"></div>

<p>Mouse over the rectangle above, and get the coordinates of your mouse pointer.</p>

<p>When the mouse is moved over the div, the p element will display the horizontal and vertical coordinates of your mouse pointer, whose values are returned from the clientX and clientY properties on the

MouseEvent object.</p>

<p id="demo"></p>

<script>

```
function myFunction(e) {  
    var x = e.clientX;  
    var y = e.clientY;  
    var coor = "Coordinates: (" + x + ", " + y + ")";  
    document.getElementById("demo").innerHTML = coor;  
}
```

```
function clearCoor() {  
    document.getElementById("demo").innerHTML = "";  
}
```

</script>

</body>

4. onmouseover()

Example:

Unit-3 Overview of Java Script

```

```

<p>The function bigImg() is triggered when the user moves the mouse pointer over the image.</p>

<p>The function normalImg() is triggered when the mouse pointer is moved out of the image.</p>

```
<script>
```

```
function bigImg(x) {  
    x.style.height = "64px";  
    x.style.width = "64px";  
}
```

```
function normalImg(x) {  
    x.style.height = "32px";  
    x.style.width = "32px";  
}
```

```
</script>
```

3.6.3.2 Keyboard Events:

1. onkeyup

Example:

<p>A function is triggered when the user releases a key in the input field. The function transforms the character to upper case.</p>

Enter your name: <input type="text" id="fname" onkeyup="myFunction()">

```
<script>
```

```
function myFunction() {  
    var x = document.getElementById("fname");  
    x.value = x.value.toUpperCase();  
}
```

```
</script>
```

2. Onkeydown

Example:

<input type="text" onkeydown="myFunction()">

```
<script>
```

```
function myFunction() {  
    alert("You pressed a key inside the input field");  
}
```

```
</script>
```

3.6.3.3 Form Events:

1. onfocus:

The onfocus event occurs when an element gets focus. The onfocus event is most often used with <input>, <select>, and <a>.

Example:

```
<body>
```

Enter your name: <input type="text" onfocus="myFunction(this)">

Unit-3 Overview of Java Script

<p>When the input field gets focus, a function is triggered which changes the background-color.</p>

```
<script>
function myFunction(x) {
  x.style.background = "yellow";
}
</script>
</body>
```

2. **onsubmit:**

The onsubmit event occurs when a form is submitted.

Example:

```
<form action="/action_page.php" onsubmit="myFunction()">
  Enter name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
<script>
function myFunction() {
  alert("The form was submitted");
}
</script>
```

3. **onblur:**

The onblur event occurs when an object loses focus.

The onblur event is most often used with form validation code (e.g. when the user leaves a form field).

Example:

Enter your name: <input type="text" id="fname" onblur="myFunction()">

<p>When you leave the input field, a function is triggered which transforms the input text to upper case.</p>

```
<script>
function myFunction() {
  var x = document.getElementById("fname");
  x.value = x.value.toUpperCase();
}
</script>
```

4. **onchange:**

The onchange event occurs when the value of an element has been changed.

For radio buttons and checkboxes, the onchange event occurs when the checked state has been changed.

Example:

```
<select id="mySelect" onchange="myFunction()">
  <option value="Audi">Audi</option>
  <option value="BMW">BMW</option>
  <option value="Mercedes">Mercedes</option>
```

Unit-3 Overview of Java Script

```
<option value="Volvo">Volvo</option>
</select>
<p>When you select a new car, a function is triggered which outputs the value of the selected
car.</p>
<p id="demo"></p>
<script>
function myFunction() {
  var x = document.getElementById("mySelect").value;
  document.getElementById("demo").innerHTML = "You selected: " + x;
}
</script>
```